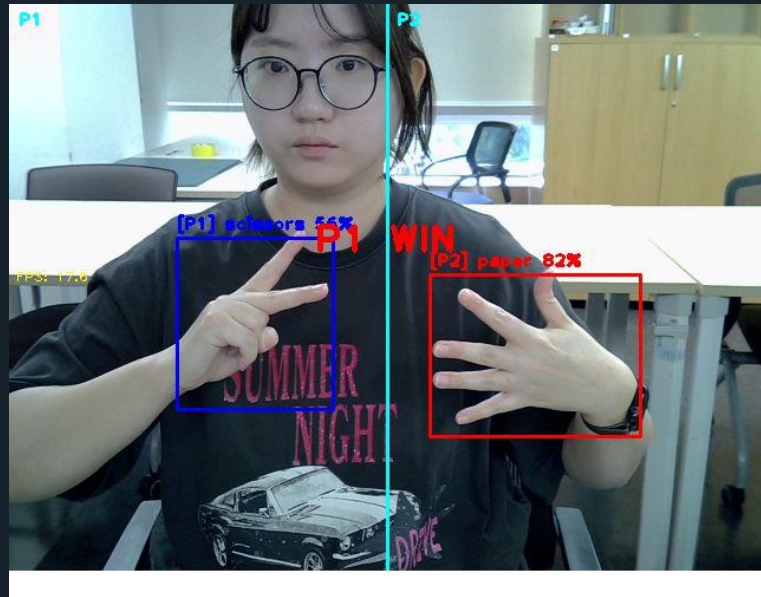


YOLO 기반 온디바이스 2인 가위바위보 판정 시스템

Raspberry Pi + LiteRT(W8A32)로 손 모양을 실시간 인식하고
승부를 판정하는 비전 게임

발표자 최현서



▲ 실제 판정 순간 캡처 (P1 scissors vs P2 paper → P1 WIN)

목차

01

주제 및 결과 요약

프로젝트 한눈에 보기

02

개발 목표 및 개발 결과

목표 대비 결과 · 실행 화면·판정 기록 · 문제 해결

03

요구 기능 구현 근거

1~5단계 실제 코드 · 실행 화면

04

모델 선정 및 성능 분석

경량화 비교 · 오류 분석 · 신뢰도 변화

05

결과 분석 및 기대 효과

원인 분석 · 기대 효과

06

향후 연구 과제 · 수행 후기

개선 → 후속 개발 → 확장 · 느낀 점

카메라 1대 · 2인 동시 인식 · 3-2-1 자동 판정

1 주제

- 수업에서 학습·경량화한 YOLO 모델(W8A32)을 엣지 보드에 올려 실제 '서비스'로 동작시키기
- 모델 출력(박스·클래스)을 게임 규칙과 연결하는 응용 로직이 핵심 과제

2 목표

- 좌/우 플레이어 구분 → 승패 판정 → 예외 처리 → 타이밍 제어 → 게임성 확장
- 5단계 체크 항목 전부 구현

3 결과

- 5단계 모두 구현 완료 (5/5)
- W8A32 모델 적용 · 보드 실측 약 9~18 FPS
- 판정 이미지·CSV 로그 자동 저장, 화면 배율 조절 추가

2 개발 목표 및 개발 결과 ①

검증 결과: 기획한 기능 5개 모두 구현

개발 환경

Raspberry Pi

USB 카메라

YOLO → LiteRT (W8A32)

OpenCV

Python · NumPy

simpleaudio

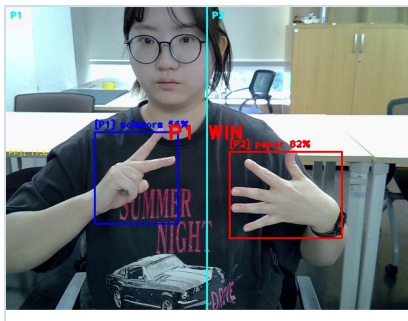
단계	구현 항목	핵심 구현	결과
1	좌·우 플레이어 구분	BB 중심 x좌표로 P1 / P2 분리	완료
2	승패 판정	클래스 조합으로 WIN / DRAW 판정	완료
3	예외 처리	손 2개·좌우 1개 조건 검사	완료
4	타이밍 제어	0.7초 안정 → 3·2·1 → 3초 고정	완료
5	게임 기능	점수·효과음·재시작·이미지/CSV 저장	완료

구현 항목 달성 5 / 5

2

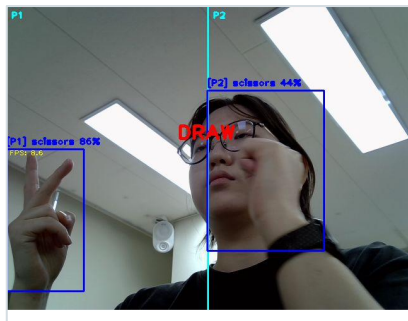
개발 목표 및 개발 결과 ② 실행 화면과 판정 기록 저장

검증 결과: 승·패·무 판정 이미지와 CSV 로그 저장 정상 동작



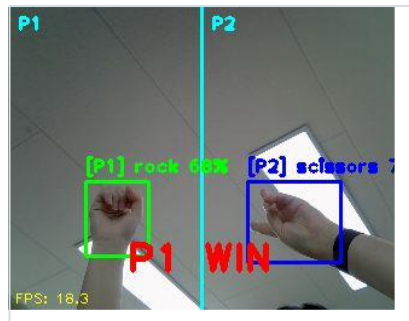
P1 scissors vs P2 paper

P1 WIN



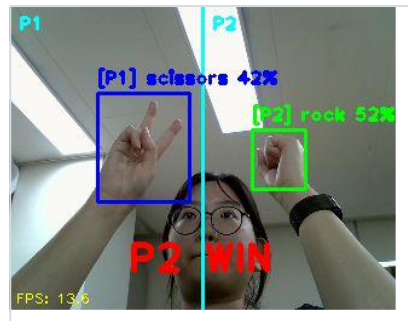
P1 scissors vs P2 scissors

DRAW



P1 rock vs P2 scissors

P1 WIN



P1 scissors vs P2 rock

P2 WIN

```

results_log.csv X EX_03_Board_RPS_PreTrained_YOLO.py 4
C: > Users > kccistc > AppData > Local > Temp > Mxt111 > RemoteFiles > 67032_4_15 > results_log.csv
1 timestamp,result,p1_class,p2_class,p1_score,draw_score,p2_score,image_path
2 20260917_124248,P1 WIN,rock,scissors,1,0,0,captures/20260917_124248_P1_WIN.jpg
3 20260917_163821,P2 WIN,scissors,rock,0,0,1,captures/20260917_163821_P2_WIN.jpg
4 20260917_164616,P1 WIN,rock,scissors,1,0,0,captures/20260917_164616_P1_WIN.jpg
5 20260918_101408,DRAW,scissors,scissors,0,1,0,captures/20260918_101408_DRAW.jpg
6 20260918_103619,P1 WIN,scissors,paper,1,0,0,captures/20260918_103619_P1_WIN.jpg
7

```

▲ saveResult()가 기록한 results_log.csv (판정 1회 = 1줄)

저장 흐름

- 판정 순간 프레임에 결과 텍스트를 합성해 captures/날짜_시각_결과.jpg로 저장
- results_log.csv에 시각·결과·클래스·누적 점수·이미지 경로를 한 줄 append
- 저장은 daemon 쓰레드로 분리 → 영상 끊김 없음

현상·원인·해결·적용 결과를 기준으로 정리

현상·원인	해결 방법	적용 결과
화면 정보 겹침 원인: 동일 표시 영역	라벨·메시지·FPS 영역 분리	정보 위치 분리
판정 시점 변동 원인: 단일 프레임 의존	0.7초 안정 후 결과 3초 고정	판정 프레임 확인 가능
결과 미보존 원인: 저장 기능 없음	이미지·CSV 자동 저장	승·패·무 기록 생성
저장 중 영상 끊김 원인: 메인 루프 I/O	저장·효과음 별도 쓰레드	영상 처리 지속
X11 화면 확대 불가 원인: 프레임 크기 고정	프레임 리사이즈 +/- 배율	0.4~3.0배 조절
한 손 중복 박스 원인: 클래스별 NMS	좌우 1개 조건·안정 대기	경고 후 판정 보류
모델 선택 필요 원인: 정확도·속도 절충	정확도 보존 우선 W8A32	mAP50-95 0.700·9~18 FPS

3

요구 기능 구현 근거 ① 플레이어 구분과 승패 판정

과제 1:2단계에 대응하는 실제 조건문

1 1단계 화면 경계

```
boundary_x =  
    frame.shape[1] // 2
```



2 Bounding Box 중심

```
center_x =  
    (x1 + x2) / 2
```



3 P1 / P2 분류

```
if center_x < boundary_x:  
    p1_detections.append(...)  
else:  
    p2_detections.append(...)
```

4 2단계 무승부

```
if p1_class == p2_class:  
    result = 'DRAW'
```



5 P1 승리 조건

```
elif p1_class == 'rock' and  
     p2_class == 'scissors':  
    result = 'P1 WIN'
```



6 나머지 경우

```
else:  
    result = 'P2 WIN'
```

judgeRPS(): DRAW 1개 · P1 승리 조합 3개 · 나머지는 P2 WIN

3

요구 기능 구현 근거 ② 손 개수 예외 처리

과제 3단계: 손이 2개가 아닐 때 판정을 중단하고 안내 메시지 출력

검출 결과 결합

양쪽 검출 목록

- `boxes = p1_detections`
- `+ p2_detections`

전체 개수 검사

2개가 아니면

- `if len(boxes) != 2:`
- `result_text =`
- `f'Need 2 hands!`
`({len(boxes)})'`

좌우 개수 검사

한쪽에 1개씩

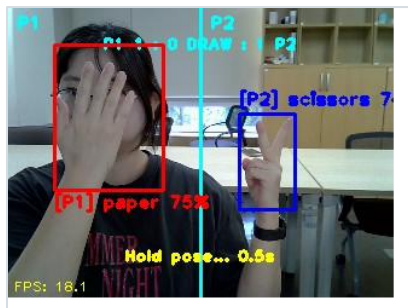
- `elif len(p1_detections)`
`!= 1`
- `or len(p2_detections)`
`!= 1:`
- `result_text = '1 hand`
`per side!'`

두 조건을 통과한 경우에만 `judgeRPS()` 실행

3

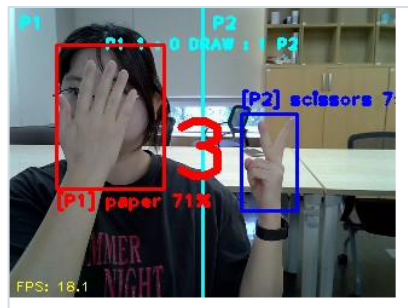
요구 기능 구현 근거 ③ 카운트다운과 프레임 고정

과제 4단계: 3-2-1 종료 순간 판정하고 FREEZE 상태로 전환



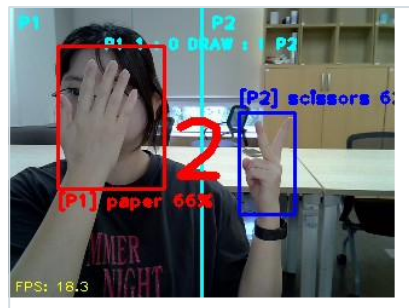
WAITING

'Hold pose... 0.5s'



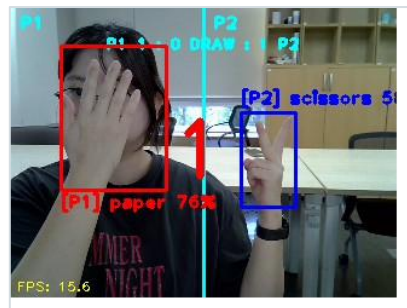
COUNTDOWN

3 (P1 paper 71%)



COUNTDOWN

2 (P1 paper 66%)



COUNTDOWN

1 (P1 paper 76%)

카운트다운

- `elapsed = time.time() - count_start`
- `if elapsed >= COUNTDOWN_SEC:`
- `count -= 1`

판정 프레임 고정

- `frozen_frame = display_frame.copy()`
- `state = 'FREEZE'`
- `freeze_start = time.time()`

3

요구 기능 구현 근거 ④ 스코어·효과음·재시작

과제 5단계에 대응하는 실제 코드

```
game loop

# 스코어 갱신
if result_text == 'P1 WIN':
    p1_score += 1
elif result_text == 'P2 WIN':
    p2_score += 1
else:
    draw_score += 1

# 효과음
playResultSound(result_text)

# 재시작 키
elif key == ord('r'):
    p1_score = 0
    p2_score = 0
    draw_score = 0
    state = 'WAITING'
```

1

스코어보드

판정 결과에 따라 P1 · DRAW · P2 점수 증가

2

승패 효과음

playResultSound()를 별도 쓰레드로 재생

3

재시작 키

r 입력 시 점수 초기화 후 WAITING 복귀

4

W8A32: mAP50-95 96% 보존 · 용량 48% 감소

모델 선정 근거 ① · Base(FP32), INT8, W8A32 검증 결과 비교

모델 (all)	P	R	mAP50	mAP50-95	용량	추론 속도
Base (PyTorch FP32)	0.936	0.966	0.993	0.727	5.4 MB	37.4 ms
INT8 LiteRT	0.950	0.927	0.989	0.678	2.88 MB	29.3 ms
W8A32 LiteRT (적용)	0.972	0.905	0.990	0.700	2.79 MB	38.2 ms

※ 동일 CPU의 수업 측정값(ms/이미지). Base는 PyTorch, 양자화 모델은 LiteRT이므로 런타임 차이가 포함됨

rock 클래스만 보면

	Base	INT8	W8A32
Recall	0.897	0.781	0.716
mAP50-95	0.637	0.595	0.623
Precision	0.975	0.971	1.000

→ 세 클래스 중 양자화에 가장 민감

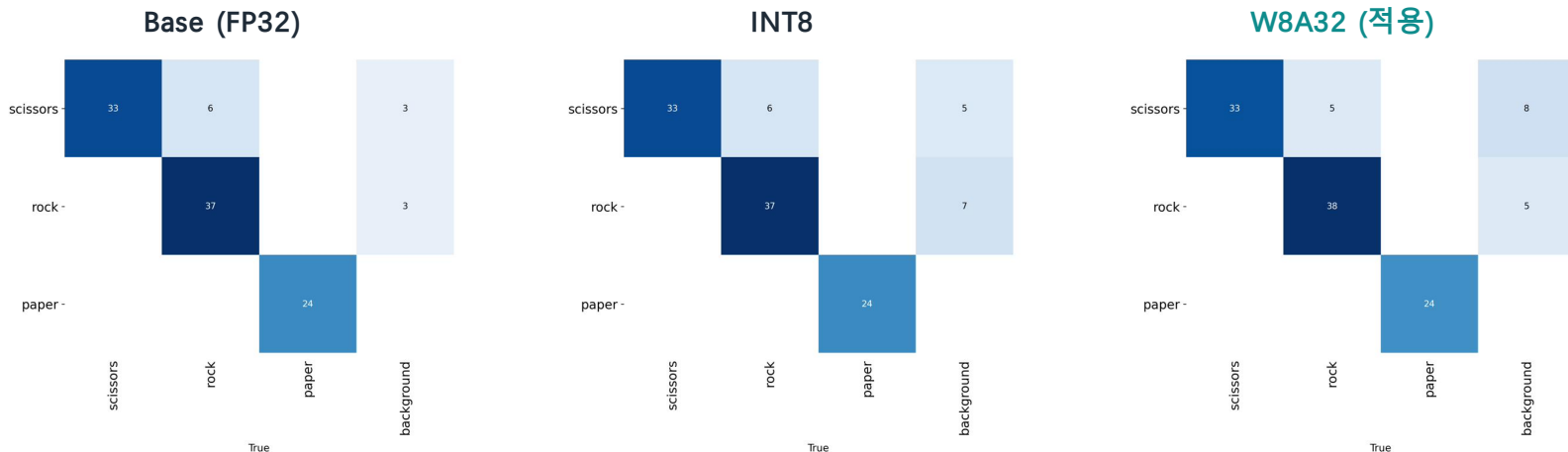
수업에서 정리한 경량화 방식

- INT8 29.3ms · 정확도 93% 보존
- W8A32 2.79MB · 정확도 96% 보존
- 선택: 정확도 보존이 높은 W8A32

4

W8A32 정분류 95/100 · 배경 오검출 13개

모델 선정 근거 ② · 혼동 행렬로 정분류와 배경 오검출 비교



항목	Base	INT8	W8A32
정분류 (/100)	94	94	95
rock → scissors 오분류	6	6	5
배경 오검출 (가짜 박스)	6	12	13

※ 배경 오검출 = 예측 scissors:rock 중 정답이 background인 칸의 합 · 비어 있는 background 예측 행은 잘라 표시

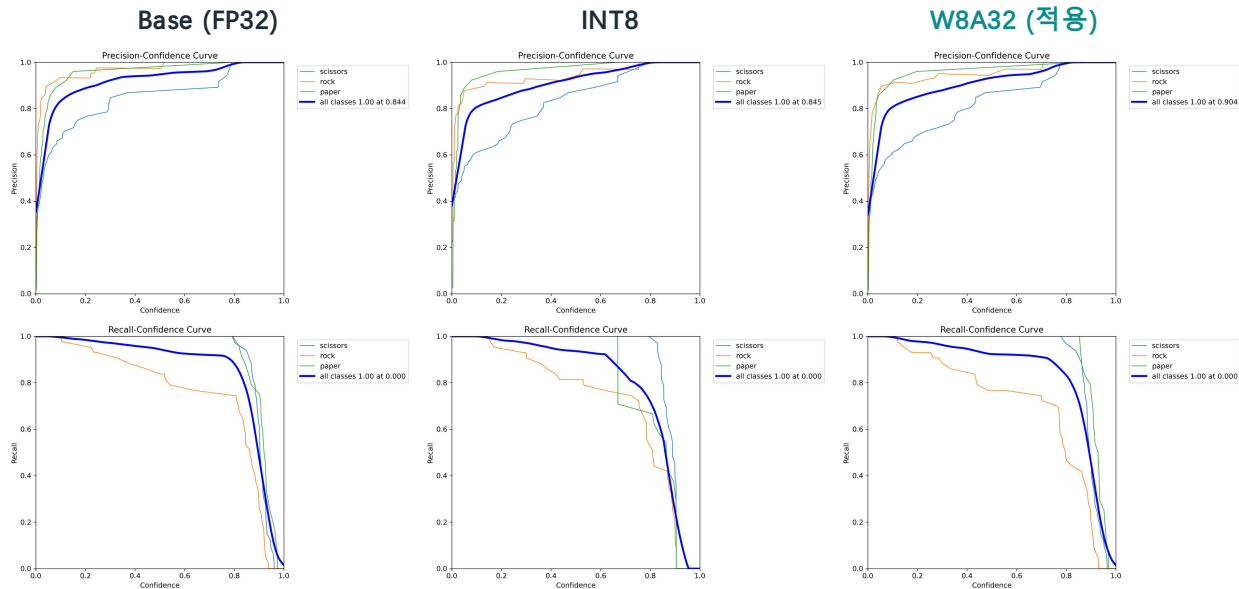
관찰

- rock → scissors가 주요 오분류
- paper는 3개 모델 모두 24/24
- 배경 오검출 6 → 12 → 13

4

CONF_TH 0.4에서 전체 Recall 약 0.95

모델 신뢰도 분석 · confidence threshold에 따른 Precision-Recall 변화



(위) Precision – Confidence · (아래) Recall – Confidence

Precision 1.0 도달 confidence

Base 0.844 INT8 0.845 W8A32 0.904

관찰

- rock Recall 먼저 감소
- CONF_TH 0.4 → Recall 약 0.95

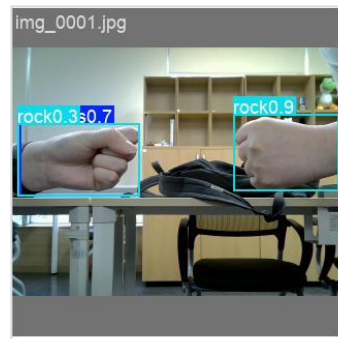
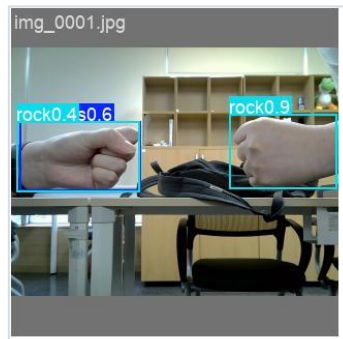
4

rock→scissors 중복 검출이 판정 오류로 연결

오분류 사례 분석 · 검증 샘플과 실제 후처리 로직의 연결



Base (FP32) — img_0001-0013-0014: 옆으로 쥔 주먹을 scissors로 예측



INT8: rock 0.4 + scissors 0.6 W8A32: rock 0.3 + scissors 0.7

내 코드와 연결해 보면

- 클래스별 NMS → 두 박스 유지
- 한쪽 2개 → 안정 대기 리셋
- W8A32: rock이 scissors로 판정 가능

5

W8A32 기반 2인 게임 구현 완료

구현 결과와 확인된 한계

5 / 5

요구 기능 구현
기획 기능 전체
완료

0.990

검증 데이터
mAP50
W8A32 · Base
0.993

9-18 FPS

보드 실측 속도
Raspberry Pi CPU

실행 중 확인한 한계

- 경계선 근처에서 P1/P2가 바뀔 수 있음
- rock→scissors 혼동과 중복 박스가 발생
- 단일 프레임 신뢰도가 66~76%로 변동
- Raspberry Pi CPU에서 약 9~18 FPS

핵심 성과

- 목표 기능 5개와 3-State 게임 흐름 구현
- 판정 이미지·CSV 로그 자동 저장
- 모델 지표와 실제 오류를 연결해 개선 방향 도출

STEP 1

즉시 개선

- N프레임 다수결
- 클래스 무관 NMS
- 좌우 최고 점수 1개

STEP 2

모델·속도 개선

- 오분류 데이터 보강
- 얼굴 배경 샘플 추가
- INT8 · 추론 스레드 검토

STEP 3

확장 실험

- NPU 가속 비교
- 키오스크 UI
- 토너먼트 모드

느낀 점

상태 관리·예외 처리의
중요성
지표와 실제 오류를 함께
분석

어려움과 극복

X11 → 화면 배율 조절
저장 지연 → 비동기 처리

마무리 소회

W8A32 온디바이스 배포
완료
다음 목표: NPU 성능 비교

감사합니다

최현서 | AI 시스템반도체 SW개발자(2기) | 프로젝트 결과보고서